UCS503P – Software Engineering Project

ResQ: Crisis Coordination Platform for Emergency Response

A Real-Time Emergency Response and Dispatch Management System

MID-SEMESTER EVALUATION REPORT

Submitted by:

1024170226 Gurnoor Singh Marok

1024170043 Rishi Gupta

1027170237 Tamanna Chadha

1024170255 Kaif Ahmad

Group: 3Q24 BE Third Year, CSE

Submitted to:

Dr. Anushka

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction 3
1.1 Background and Context 3
1.1.1 Problem Statement 3
1.2 Project Objectives 3
2 Methodology and Design 4
2.1 System Architecture 4
2.1.1 High-Level Design 4
2.1.2 Technical Stack 4
2.2 UML Diagrams 4
2.2.1 Use Case Diagram 5
2.2.2 Class Diagram 6
2.2.3 Sequence Diagram 7
2.2.4 Activity Diagram 8
2.3 Implementation Details 9
2.3.1 Module 1: Core Functionality 9
2.3.2 Module 2: Secondary Features 9
3 Results and Evaluation 10
3.1 Testing Methodology 10
3.2 Performance and Verification Metrics 10
3.3 Analysis 10
4 Conclusion 11
4.1 Summary of Work 11
4.2 Key Findings 11
4.3 Future Work and Recommendations 11
4.4 Final Remarks 11
Bibliography 12

Chapter 1

Introduction

0.1 1.1 Background and Context

Emergency and disaster response is fundamentally a coordination problem before it is a technology problem. During a local emergency, the difficulty is rarely in collecting a report: a form and a map pin solve that initial part of the problem. The more difficult task is coordinating independent actors who hold partial information and may have different priorities, including affected people, volunteers, non-governmental organizations (NGOs), resource providers, and administrators.

Response capacity during a crisis is scarce. A request that receives duplicate attention while another remains unseen represents capacity that has not been used effectively. ResQ addresses this gap through a shared operational view of requests, responders, locations, and status.

0.1.1 1.1.1 Problem Statement

Existing ad hoc approaches to local emergency coordination, such as phone calls, spreadsheets, messaging groups, and disconnected organizational dashboards, exhibit several recurring failure modes:

- **Duplicated or wasted effort:** Two responders may act on the same request while another request remains unanswered because the actors cannot reliably observe one another's claims.
- **Fragmented visibility:** Different roles may work from separate views, making it difficult to maintain a synchronized understanding of request status.
- **Unmatched partial resources:** Some emergency needs require combining contributions from multiple providers; simple request/response record-keeping does not model this well.
- **Absence of an accountability loop:** A responder should not be the only party deciding that assistance has actually been received.
- **Location and contact privacy risk:** Publishing exact coordinates or contact information of affected people can expose them unnecessarily.

0.2 1.2 Project Objectives

For the mid-semester evaluation, the project focuses on a demonstrable core that establishes the coordination model and the security/privacy foundations needed for later real-time features.

1. Implement a server-enforced five-stage request lifecycle: Unverified → Verified → In Progress → Resolved → Closed, with role-based authorization at every transition.
2. Build a unified map-and-log dispatch console that can be used by the five project roles from one codebase, with the available actions adapting to the logged-in role.
3. Protect affected-person location and contact privacy through coordinate fuzzing and authorization-aware response redaction.
4. Provide public emergency submission and request tracking without requiring an account for the initial report, while maintaining a free/open-source or free-tier technology stack.

Chapter 2

Methodology and Design

0.3 2.1 System Architecture

0.3.1 2.1.1 High-Level Design

ResQ follows a conventional three-tier web architecture. The React client communicates with a stateless Express REST API, while the API is the only application component that communicates with MongoDB. The database is treated as the source of truth for user and request state.

The central business rule is a forward-only request state machine. Supporting concerns include authorization, privacy redaction, and a status-history audit trail. The frontend uses one shared map-and-rail console rather than five visually separate applications; only the actions available to each role change.

Socket.IO-based real-time propagation and the rule-based matching engine are intentionally treated as later-phase capabilities. This keeps the mid-semester prototype focused on the core lifecycle, privacy, submission, tracking, and dispatch-console functionality.

0.3.2 2.1.2 Technical Stack

Layer	Technology	Purpose / Constraint
Frontend	React 18 + Vite + TypeScript + Tailwind CSS	Responsive SPA and design-system-based UI
Backend	Node.js + Express + TypeScript	REST API, authentication, lifecycle and authorization logic
Database	MongoDB + Mongoose	Persistent user/request data and status history
Authentication	JWT + bcrypt	Role-aware authentication without a paid identity vendor
Maps	react-leaflet + OpenStreetMap	Map-based request visualization without a billed map provider
Testing / CI	Vitest, Supertest, GitHub Actions	Automated lifecycle, authorization and API verification
Future real-time layer	Socket.IO	Planned cross-dashboard state propagation

0.4 2.2 UML Diagrams

The UML models below describe the current ResQ prototype. The Use Case diagram includes explicit «`include`» and «`extend`» relationships, while the Class, Sequence, and Activity diagrams are aligned with the same roles, request lifecycle, and privacy-aware architecture.

0.4.1 2.2.1 Use Case Diagram

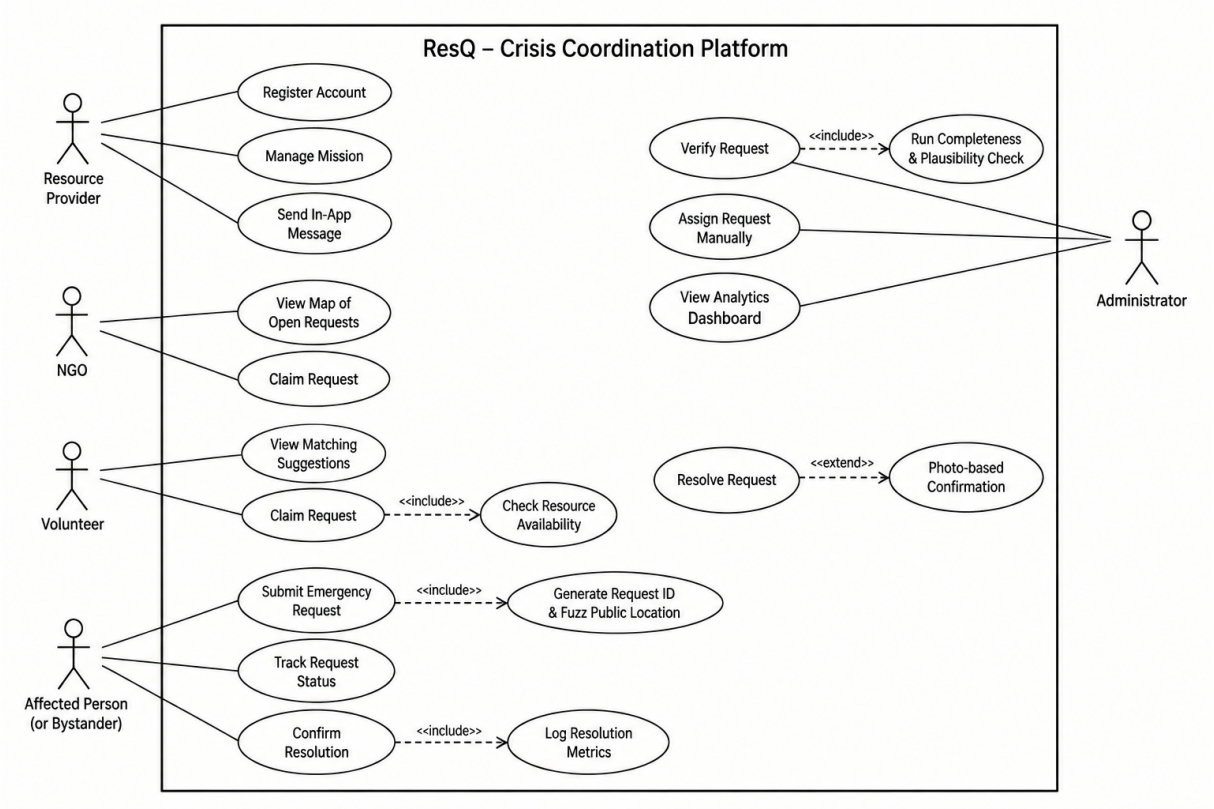


Figure 1: *ResQ use case diagram*

0.4.2 2.2.2 Class Diagram

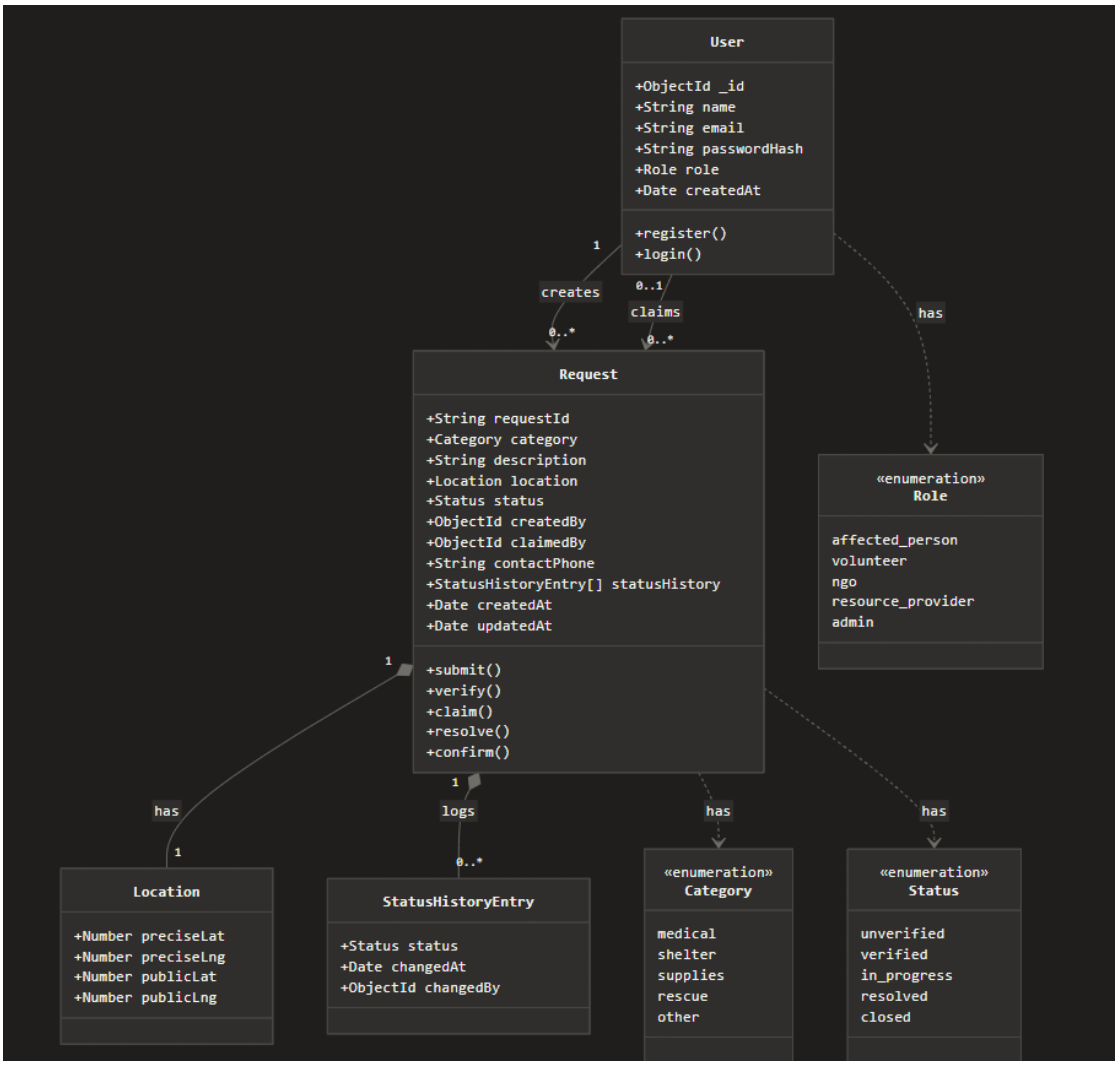


Figure 2: *ResQ class diagram*

0.4.3 2.2.3 Sequence Diagram

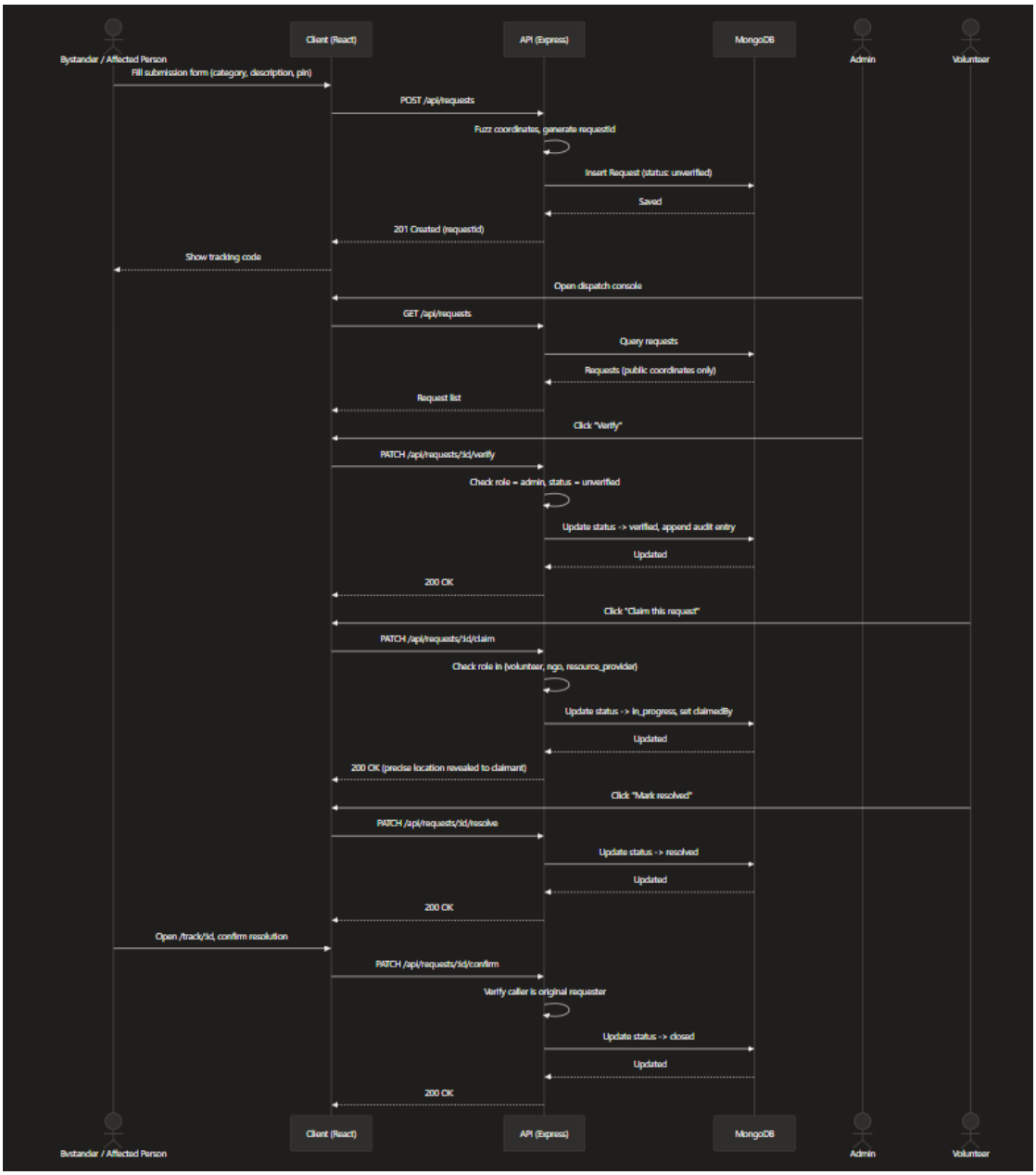


Figure 3: *Request submission, retrieval, and responder claim sequence.*

0.4.4 2.2.4 Activity Diagram

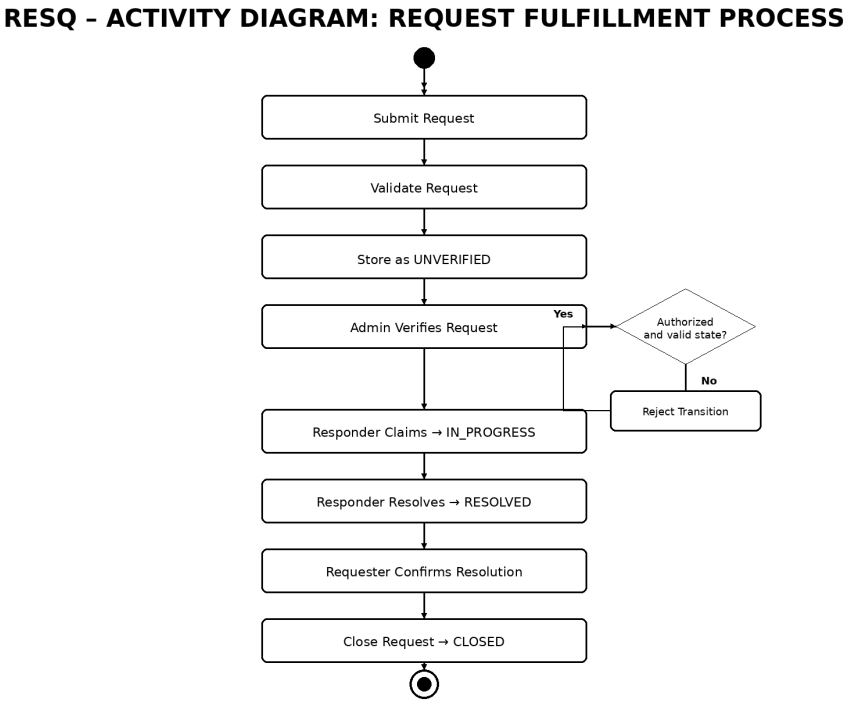


Figure 4: *Request fulfillment activity flow with authorization/state validation.*

0.5 2.3 Implementation Details

0.5.1 2.3.1 Module 1: Core Functionality

The User model represents the five project roles: affected person, volunteer, NGO, resource provider, and administrator. Passwords are stored as bcrypt hashes rather than plaintext values. The Request model tracks the request identifier, category, description, status, creator/claimant references, location, and status history.

Location privacy is enforced structurally. Each request contains a precise coordinate pair and a separate fuzzed public coordinate pair. API response sanitization determines whether precise coordinates and contact information may be returned to the caller. Unauthorized callers receive only the public location.

The lifecycle is centralized in transition logic. Each transition checks both the current state and the authorization requirements, preventing skipped or reversed states. Closing a request requires confirmation by the original requester, with the anonymous-request path using the request identifier and phone verification.

0.5.2 2.3.2 Module 2: Secondary Features

Authentication uses hand-rolled JWT-based sessions and bcrypt password hashing. Public registration is available for project roles except administrator; administrator accounts are provisioned separately.

The frontend provides a shared map-and-rail console. The Leaflet map displays non-closed requests, while the dispatch rail provides filtering and role-specific actions. A mobile-width layout adapts the map/log presentation so that the same workflow remains usable on smaller screens.

The public workflows allow an affected person or bystander to submit a request without logging in and receive a tracking identifier immediately. A tracking page can then display the current lifecycle stage and support the confirmation step. Seed data is used to make the prototype demonstrable without requiring all requests to be entered manually during a presentation.

Chapter 3

Results and Evaluation

0.6 3.1 Testing Methodology

- **Unit and integration testing:** Backend helpers and API behavior are tested using Vitest and Supertest against an in-memory MongoDB instance where applicable.
- **Lifecycle verification:** Tests exercise the complete request lifecycle and explicitly cover stage skipping, unauthorized role actions, and attempts to force-close a request.
- **Manual walkthrough:** The demonstration flow of submission, verification, claiming, resolution, and confirmation is checked across the project roles.
- **Not yet performed:** Formal Playwright end-to-end browser testing, concurrent load testing, and pilot testing with real or simulated end users remain future work.

0.7 3.2 Performance and Verification Metrics

The evaluation below separates properties that are currently verified from metrics that require capabilities or pilot conditions not yet available. This avoids presenting projected values as achieved measurements.

Metric	Target / Requirement	Status at Mid-Semester
Lifecycle transition correctness	All defined transitions enforced; illegal transitions rejected	Achieved – verified by automated tests
Location / contact privacy	No unauthorized exposure of precise coordinates or phone numbers	Achieved – verified by automated tests
Role-based authorization	Actions restricted to intended roles at protected endpoints	Achieved – verified by automated tests
Time-to-Match	Median $\leq$ 15 minutes during pilot	Pending – requires pilot and matching workflow
Cross-dashboard propagation	$<$ 2 seconds	Not yet applicable – Socket.IO real-time layer is future work
Double-booking rate	0%	Not yet measurable – requires concurrent-user pilot

0.8 3.3 Analysis

The current checkpoint establishes the core implementation objectives: a server-enforced lifecycle, a unified map-and-log console, and privacy-by-design. The project also maintains the intended free/open-source or free-tier technology constraint.

A key implementation decision is to centralize lifecycle transitions rather than duplicate state checks across individual endpoints. This reduces the chance that a later route bypasses authorization or lifecycle rules. The statusHistory audit trail also provides a foundation for later accountability analysis.

Anonymous submission requires an additional identity-verification path because the requester may not have an account. The request identifier plus phone verification provides the confirmation path without requiring account registration.

Chapter 4

Conclusion

0.9 4.1 Summary of Work

At this checkpoint, ResQ establishes the core data model, a server-enforced five-stage request lifecycle, role-based authorization, privacy-aware location handling, a unified map-and-log console, public request submission and tracking workflows, and automated backend verification. The implementation remains consistent with the phased development plan and is suitable as a demonstrable mid-semester prototype.

0.10 4.2 Key Findings

- Centralizing status-transition logic is essential for enforcing lifecycle order and authorization consistently.
- A shared console across five roles reduces duplicated UI structure while allowing actions to remain role-specific.
- Privacy must be implemented at the API/data boundary rather than relying only on frontend hiding.
- Anonymous submission requires an explicit verification path if the affected person must later confirm resolution.

0.11 4.3 Future Work and Recommendations

1. Implement Socket.IO-based real-time cross-dashboard propagation so that state changes are reflected without manual refresh.
2. Implement the rule-based matching and mission-management functionality for combining partial resource contributions.
3. Add in-app messaging between coordinating actors for claimed requests.
4. Build administrator analytics and evaluate an optional, decoupled AI-assisted classification module.
5. Add Playwright end-to-end tests and complete free-tier deployment for a live demonstration.
6. Conduct a pilot with real or simulated users to measure Time-to-Match, propagation latency, and double-booking rate.

0.12 4.4 Final Remarks

The work completed so far validates the core architectural direction of ResQ: a server-enforced state machine, privacy by design, and a shared operational console. The remaining work is substantial but follows directly from the phased plan established at the proposal stage. The UML models included in this report have been kept consistent with that current architecture so that the documentation describes the system being built rather than unrelated future services.

Bibliography

[1] LaTeX Project Team. *LaTeX2e: An updated version of LaTeX*. 2024.

[2] Meta Open Source. *React Documentation*. 2024.

[3] OpenJS Foundation. *Express – Node.js web application framework*. 2024.

[4] MongoDB, Inc. *MongoDB Atlas Documentation*. 2026.

[5] Leaflet. *Leaflet – a JavaScript library for interactive maps*. 2024.

[6] M. Jones, J. Bradley, N. Sakimura. *JSON Web Token (JWT)*. RFC 7519, Internet Engineering Task Force, 2015.

[7] Vitest. *Vitest – A Vite-native testing framework*. 2024.

[8] OWASP Foundation. *OWASP Top Ten*. 2024.